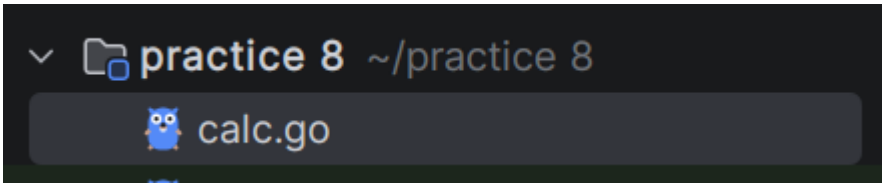
Before start, please make sure you've read the following materials:

- Go Proverbs
- Unit-testing in Go. A Beginners Guide
- 4 Mocking Approaches for Go
- Testing Pyramid
- How to write useful tests for microservices
- Integration tests with Docker and Go
- Writing functional testing in Go

Step 1: Beginner Start

Create a directory (`practice 8` in my case)

Inside this directory, create `calc.go` file:



`calc.go`:

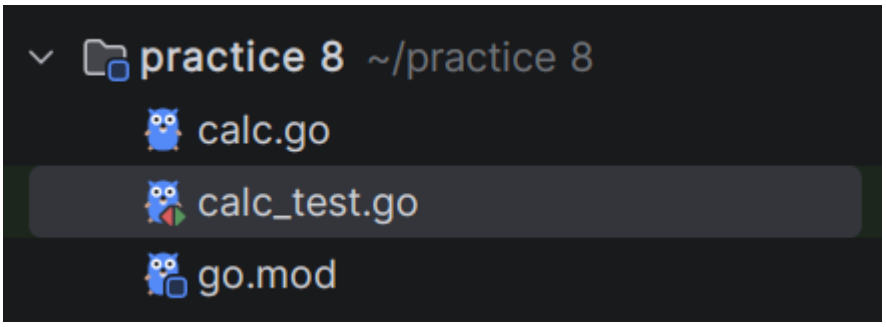
```go
package main

func Add(a, b int) int {
    return a + b
}
```

Let's write test for this function:

In the same directory, create `calc_test.go` file

`Note`: In Go, test files must end with `_test.go` to be recognized by the testing tool.



Inside `calc_test.go`, you write a test function: `TestAdd` func

```go
package main

import "testing"

func TestAdd(t *testing.T) {}
```

The function name starts with `Test` and takes a single `*testing.T` parameter. Go automatically discovers and runs any function that follows this convention.

Let's continue:

```go
package calc

import "testing"

func TestAdd(t *testing.T) {
    got := Add(2, 3)
    want := 5
    if got != want {
        t.Errorf("Add(2, 3) = %d; want %d", got, want)
    }
}
```

- The `t.Errorf` call reports a test failure. Unlike some frameworks, Go doesn't provide special assertions – you simply check a condition and call `t.Errorf` or `t.Fatalf` if it fails.
- Each test is a standalone function. You can write as many as you like, and Go will run them all.

Step 2: Running your test

```
go test
```

Note:

This runs tests for the current package (files ending with `_test.go`). If you want to run tests recursively in all subdirectories of your project, use:

```
go test ./...
```

The `./...` pattern is shorthand for "run tests in this directory and all subdirectories". This is especially useful in larger projects where your code is spread across multiple packages.

If everything is working, you should see output indicating that the test passed:

```
bakytzhan@Bakytzhan:~/practice 8$ go test
PASS
ok      practice-8      0.005s
```

You can add the `-v` flag for verbose output:

```
go test -v
```

This will show you the names of the tests as they run:

```
bakytzhan@Bakytzhan:~/practice 8$ go test -v
=== RUN   TestAdd
--- PASS: TestAdd (0.00s)
PASS
ok      practice-8      0.005s
```

Not much difference for a single test, but it becomes useful as you add more tests.

Now let's see what happens if the test fails. Change the expected value in `calc_test.go` to an incorrect one:

```
...
  want := 6 // Incorrect expected value
...
```

Run the test again:

```
bakytzhan@Bakytzhan:~/practice 8$ go test
--- FAIL: TestAdd (0.00s)
    calc_test.go:9: got 5, want 6
FAIL
exit status 1
FAIL    practice-8      0.006s
```

or with verbose output:

```
bakytzhan@Bakytzhan:~/practice 8$ go test -v
=== RUN   TestAdd
    calc_test.go:9: got 5, want 6
--- FAIL: TestAdd (0.00s)
FAIL
exit status 1
FAIL    practice-8      0.005s
```

Of course, your tests should always check for the correct expected values! A failing (but correct) test is a sign that your code needs to be fixed.

We only created one test file and one test function with one assertion here, but Go's testing tool can handle many files and functions at once. Behind the scenes, Go will automatically:

- Find all `_test.go` files in the specified packages (for example, current directory for `go test`, or recursively in all subdirectories with `go test ./...`).
- Identify functions that start with `Test` and have the correct signature.

- Compile them together with your package into a temporary test binary.
- Execute each test function and report the results.

`t.Errorf` vs `t.Fatalf`

In the example above, we used `t.Errorf` to report test failures. This function logs the error but allows the test to continue running. This is useful when you want to check multiple conditions in a single test function.

In contrast, `t.Fatalf` logs the error and immediately stops the execution of the current test. Use `t.Fatalf` when continuing the test after a failure doesn't make sense or could cause misleading results.

While `t.Errorf` and `t.Fatalf` use `fmt`-style formatting, for simple messages without formatting, you can also use `t.Error` and `t.Fatal`, respectively.

Table-Driven Tests

In Go, it's common to want to run the same test logic for multiple inputs and expected outputs. Rather than writing a separate test function for each case, Go developers often use table-driven tests. This pattern keeps your tests concise, readable, and easy to extend.

Table-Driven `Add` Test

Let's rewrite our Add test using a table-driven approach:

```go
// calc_test.go
package main

import "testing"

func TestAddTableDriven(t *testing.T) {
    tests := []struct {// Define a struct for each test case and create a
slice of them
```

```go
        name string
        a, b int
        want int
    }{
        {"both positive", 2, 3, 5},
        {"positive + zero", 5, 0, 5},
        {"negative + positive", -1, 4, 3},
        {"both negative", -2, -3, -5},
    }

    for _, tt := range tests {// Loop over each test case
        t.Run(tt.name, func(t *testing.T) {// Run each case as a subtest
            got := Add(tt.a, tt.b)
            if got != tt.want {// Check the result
                t.Errorf("Add(%d, %d) = %d; want %d", tt.a, tt.b, got,
tt.want) // Report failure if it doesn't match
            }
        })
    }
}
```

Here's how it works:

- We define a slice of structs, each representing a test case.
- Each struct contains the test name, input values, and the expected result.
- We loop over the slice and call `t.Run(tt.name, func(t *testing.T) { ... })` to run each test as a subtest.
- If a subtest fails, you can see which one by its name in the output.

```
bakytzhan@Bakytzhan:~/practice 8$ go test -v
=== RUN   TestAddTableDriven
=== RUN   TestAddTableDriven/both_positive
=== RUN   TestAddTableDriven/positive_+_zero
=== RUN   TestAddTableDriven/negative_+_positive
=== RUN   TestAddTableDriven/both_negative
--- PASS: TestAddTableDriven (0.00s)
    --- PASS: TestAddTableDriven/both_positive (0.00s)
    --- PASS: TestAddTableDriven/positive_+_zero (0.00s)
    --- PASS: TestAddTableDriven/negative_+_positive (0.00s)
    --- PASS: TestAddTableDriven/both_negative (0.00s)
PASS
ok      practice-8      0.006s
```

Step 3: Let's test some of the production code

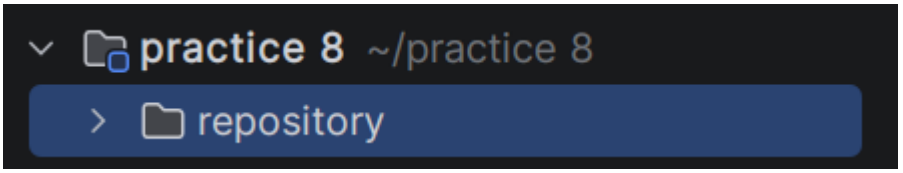
One day, you will start to write unit tests for your production code. One of the powerfull approaches that developers use is `mocking`

Mocking is an essential aspect of unit testing. It allows you to isolate the code you are testing by replacing dependencies with mock implementations. In doing so, it allows you to simulate the behavior of complex or external dependencies. In Go, there are several ways in which we can mock application behavior.

There are 2 ways to create mocks: manual and framework-based

We will consider a framework approach.

In the same directory create `repository` dir:



Inside `repository` create `user_repository.go` file:

Define `User` struct and `UserRepository` interface:

```
package repository

type User struct {
    ID   int
    Name string
}

type UserRepository interface {
    GetUserByID(id int) (*User, error)
    CreateUser(user *User) error
}
```

Install the required dependencies:

Install Go Mock, the **mockgen** tool,
and **stretchr/testify** package. The `mockgen` package is used

to generate mock code. It works by taking an interface defined in your code as input and generates a corresponding mock object.

In our case, the `mockgen` package will be used to generate the `MockUserRepository` struct and its methods. The `stretchr/testify` is used to enhance the test assertions. In our case, it does that by checking if the `UserService` functions behave as expected.

Run the following commands:

```
go get go.uber.org/mock/gomock
go install go.uber.org/mock/mockgen@latest
go get github.com/stretchr/testify/assert
```

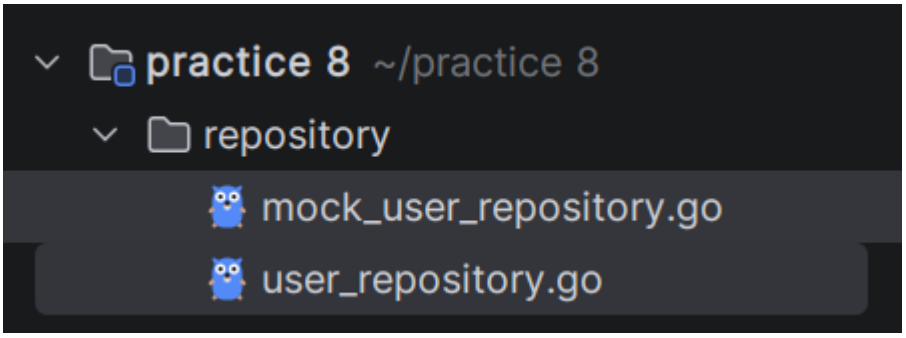
Generate the mock

Now, you'll use the mockgen tool to generate mocks for your interface by running the following command in your terminal:

```
mockgen -source=repository/user_repository.go -package=repository >
repository/mock_user_repository.go
```

```
bakytzhan@Bakytzhan:~/practice 8$ mockgen -source=repository/user_repository.go -package=repository > repository/mock_user_repository.go
```

Your `repository` dir should look like this after generating mocks:



calc.go | calc_test.go | user_repository.go | mock_user_repository.go

Generated source files should not be edited. The changes will be lost when sources are regenerated.

```go
// Code generated by MockGen. DO NOT EDIT.
// Source: repository/user_repository.go
//
// Generated by this command:
//
//  mockgen -source=repository/user_repository.go -package=repository
//

// Package repository is a generated GoMock package.
package repository

import (
    reflect "reflect"

    gomock "go.uber.org/mock/gomock"
)

// MockUserRepository is a mock of UserRepository interface.
Implement interface
type MockUserRepository struct {  10 usages
    ctrl     *gomock.Controller
    recorder *MockUserRepositoryMockRecorder
    isgomock struct{}
}
```

Define the UserService

Next, you'll need to create the `service` package and define the `UserService` struct inside it. So, first, create a new directory named *service*:

```
mkdir service
```

Then, create a new file named `user_service.go` inside the `service` directory, and write the code below into it.

```go
package service

import "mock_go/repository"

type UserService struct {
    repo repository.UserRepository
}

func NewUserService(repo repository.UserRepository) *UserService {
    return &UserService{repo: repo}
}

func (s *UserService) GetUserByID(id int) (*repository.User, error) {
```

```
    return s.repo.GetUserByID(id)
}

func (s *UserService) CreateUser(user *repository.User) error {
    return s.repo.CreateUser(user)
}
```

Write unit-tests using the mocks:

To write the unit test using mocks, you'll need a test file to test `UserService`. So, create a new file named `user_service_test.go` inside the *service* directory. Then paste the code below into the new file:

```
package service

import (
    "practice-8/repository"
    "testing"

    "github.com/stretchr/testify/assert"
    "go.uber.org/mock/gomock"
)

func TestGetUserByID(t *testing.T) {
    ctrl := gomock.NewController(t)
    defer ctrl.Finish()

    mockRepo := repository.NewMockUserRepository(ctrl)
    userService := NewUserService(mockRepo)

    user := &repository.User{ID: 1, Name: "Bakytzhan Agai"}
    mockRepo.EXPECT().GetUserByID(1).Return(user, nil)

    result, err := userService.GetUserByID(1)
    assert.NoError(t, err)
    assert.Equal(t, user, result)
}

func TestCreateUser(t *testing.T) {
    ctrl := gomock.NewController(t)
    defer ctrl.Finish()

    mockRepo := repository.NewMockUserRepository(ctrl)
    userService := NewUserService(mockRepo)

    user := &repository.User{ID: 1, Name: "Bakytzhan Agai"}
    mockRepo.EXPECT().CreateUser(user).Return(nil)

    err := userService.CreateUser(user)
```

```
    assert.NoError(t, err)
}
```

Run the tests:

```
go test -v ./...
```

```
bakytzhan@Bakytzhan:~/practice 8$ go test -v ./...
=== RUN   TestAddTableDriven
=== RUN   TestAddTableDriven/both_positive
=== RUN   TestAddTableDriven/positive_+_zero
=== RUN   TestAddTableDriven/negative_+_positive
=== RUN   TestAddTableDriven/both_negative
--- PASS: TestAddTableDriven (0.00s)
    --- PASS: TestAddTableDriven/both_positive (0.00s)
    --- PASS: TestAddTableDriven/positive_+_zero (0.00s)
    --- PASS: TestAddTableDriven/negative_+_positive (0.00s)
    --- PASS: TestAddTableDriven/both_negative (0.00s)
PASS
ok      practice-8      (cached)
?       practice-8/repository   [no test files]
=== RUN   TestGetUserByID
--- PASS: TestGetUserByID (0.00s)
=== RUN   TestCreateUser
--- PASS: TestCreateUser (0.00s)
PASS
ok      practice-8/service      0.003s
```

Nice Job!

Individual Assignment (3 points)

1. Implement `Divide` and `Subtract` func that receives two integer numbers and proceeds arithmetical operations.

- Write unit test for `Divide` func handling both success and error cases. (e.g. division by zero)

- Write unit test for handling:

- Both positive numbers
- Positive minus zero
- Negative minus positive
- Both negative

> For `Subtract`

- Demonstrate the table-driven testing

---

2. Expand the `UserRepository` interface from `step 3`:

Add new methods to `UserRepository` interface:

```
GetByEmail(email string) (*User, error)
UpdateUser(user *User) error
```

```
DeleteUser(id int) error
```

2.1 Generate mocks with `mockgen` command

2.2 Expand `user_service.go` by adding new logic:

```
...

func (s *UserService) RegisterUser(user *repository.User, email string)
error {
    existing, err := s.repo.GetByEmail(email)
    if existing != nil {
        return fmt.Errorf("user with this email already exists")
    }

    if err != nil {
        return fmt.Errorf("error getting user with this email")
    }
    return s.repo.CreateUser(user)
}

func (s *UserService) UpdateUserName(id int, newName string) error {
    if newName == "" {
        return fmt.Errorf("name cannot be empty")
    }

    user, err := s.repo.GetUserByID(id)
    if err != nil {
        return err
    }

    user.Name = newName
    return s.repo.UpdateUser(user)
}

func (s *UserService) DeleteUser(id int) error {
    if id == 1 {
        return errors.New("it is not allowed to delete admin user")
    }

    return s.repo.DeleteUser(id)
}
```

Mandatory test cases:

`RegisterUser`:

1. User already exists

2. New User -> Success

3. Repository error on `CreateUser`

`UpdateUserName`

1. Empty name

2. User not found/repo error

3. Successful update

4. UpdateUser Fails

> Verify that name was actually changed before update

`DeleteUser`

1. Attempt to delete admin

2. Successfull delete

3. Repository Error

> Verify that user was actually deleted

---

3. Test `GetRate` method that requests external API for getting exchange rates.

Source code(DO NOT EDIT):

```
package main

import (
	"encoding/json"
	"fmt"
	"net/http"
	"time"
)

type RateResponse struct {
	Base     string  `json:"base"`
	Target   string  `json:"target"`
	Rate     float64 `json:"rate"`
	ErrorMsg string  `json:"error,omitempty"`
}

type ExchangeService struct {
	BaseURL string
	Client  *http.Client
}

func NewExchangeService(baseURL string) *ExchangeService {
```

```go
    return &ExchangeService{
        BaseURL: baseURL,
        Client: &http.Client{
            Timeout: 5 * time.Second,
        },
    }
}

// GetRate requests the rate. Example URL: /convert?from=USD&to=EUR
func (s *ExchangeService) GetRate(from, to string) (float64, error) {
    url := fmt.Sprintf("%s/convert?from=%s&to=%s", s.BaseURL, from, to)

    resp, err := s.Client.Get(url)
    if err != nil {
        return 0, fmt.Errorf("network error: %w", err)
    }
    defer resp.Body.Close()

    var result RateResponse
    if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {
        return 0, fmt.Errorf("decode error: %w", err)
    }

    fmt.Println(result)

    if resp.StatusCode != http.StatusOK {
        if result.ErrorMsg != "" {
            return 0, fmt.Errorf("api error: %s", result.ErrorMsg)
        }
        return 0, fmt.Errorf("unexpected status: %d", resp.StatusCode)
    }

    return result.Rate, nil
}
```

To solve this, use the standard `net/http/httptest` package. The basic idea is to replace the real `BaseURL` with the URL of a temporary test server.

- **net/http/httptest: official documentation**

Student should test:

- **Successfull scenario**
- **API Business Error:**
  - **Condition: The server returns a 404 Not Found or 400 Bad Request status and JSON of the form {"error": "invalid currency pair"}.**
- **Malformed JSON:**

    - Condition: The server returns text that is not valid JSON (for example, Internal Server Error or just a truncated string).
- Slow Response/Timeout
- Server Panic / 500 Internal Server Error
- Empty Body

It is prohibited to ignore closing the mock-server!

Demo video scenario

- Student runs all tests separately for each task(1, 2, and 3) and examples from steps 1, 2, and 3 using a `-v` flag.

- Student demonstrates the result of execution of each test from terminal

Definition of Done (DOD)

- All tasks are accomplished according to the instructions, and all the examples from steps 1, 2, and 3 are implemented by student
- All tests executes with a `pass` status
- `Demo video` is recorded according to the `Demo video scenario`

Deliverables:

- Push all the changes to your GitHub repository
- Submit the link to this repository(Please, submit the link as a LINK ATTACHMENT, not a file)
- Submit [1-2]-minute demo video of your project as a VIDEO-FILE ATTACHMENT. You can speed it up if it is too long.

> ## Below are marking criteria:

Criteria	Weight
Problem 1	0.5 points

Criteria	Weight
Problem 2	1.0 points
Problem 3	1.5 points
OVERALL	3 points

Below are penalties criteria:

Adherence to all the requirements above is mandatory. Failure to implement any of the specified points (e.g., missing `sync.WaitGroup` or neglecting `context` support) will result in a significant point deduction.

Category	Criteria	Penalty
Video Demo	Video Demo is does not Present or Unable to Open/View or Contradicts ethical standards or Contradicts the Demo video scenario	3 points
Video Demo Link	The video demo is specified in any way other than the `video-file` attachment	0.5 points
Git Repo Link	The link to the git repository is specified in any way other than the `link-attachment`	0.5 points
Git Repo Content	The content of git repository for this practice is corrupted or unable to Open/Read	3 points
Your Task part	At least one requirement (e.g. Demonstrate the execution of the code using -race flag.) from `Individual Assignment` part is failed to meet	100% of a problem's solution mark
Cheating/ Malpractice	Students practicing cheating, duplicating the contents of other students solutions	100% of the `Overall` mark for each of the students
Definition of Done	At lease one criteria from `Definition of Done` is failed to meet	100% of the `Overall` mark

---

DEADLINE: 19.04.2026, SUNDAY 23:59

OPTIONAL FEATURES (Choose Freely)

> Students may implement any number of the following to increase difficulty and engagement. Your optional features will be considered only if you demonstrated them in your demo video and explained it in details.

🐣 EASY: "The Baby Gopher"

1. Table-Driven tests everywhere

Not only for `Subtract` but also:

- `RegisterUser`
- `UpdateUserName`
- `DeleteUser`

2. Use `t.Run()` subtests:

```
t.Run("user exists", func(t *testing.T) { ... })
```

3. Use `testify/assert` AND `require`

👨‍💻 MEDIUM: "The Gopher-at-Work"

1. Coverage analysis

Create coverage report using

- `go test -cover {???}`
- `go test -coverprofile={???}`
- Generate html report

After getting the coverage report, achieve at least 80% coverage of the code with your tests.

Example:

Файл /tmp/cover3970469264/coverage.html#file0

Gmail Teams Outlook WhatsApp Web Telegram Перевести PGE ATTENDANCE ЭКЗАМЕНЫ Трен. Янд. Dis

practice-8/service/user_service.go (42.1%) not tracked not covered covered

```
        return &UserService{repo: repo}
}

func (s *UserService) GetUserByID(id int) (*repository.User, error) {
        return s.repo.GetUserByID(id)
}

func (s *UserService) CreateUser(user *repository.User) error {
        return s.repo.CreateUser(user)
}

func (s *UserService) RegisterUser(user *repository.User, email string) error {
        existing, err := s.repo.GetByEmail(email)
        if existing != nil {
                return fmt.Errorf("user with this email already exists")
        }

        if err != nil {
                return fmt.Errorf("error getting user with this email")
        }
        return s.repo.CreateUser(user)
}

func (s *UserService) UpdateUserName(id int, newName string) error {
        if newName == "" {
                return fmt.Errorf("name cannot be empty")
        }

        user, err := s.repo.GetUserByID(id)
        if err != nil {
                return err
        }

        user.Name = newName
        return s.repo.UpdateUser(user)
}
```

🧙 ADVANCED: "The Gopher Wizard"

CI/CD Pipeline with Automated Testing

Set up an automated pipeline that runs tests on every change in the git repository and ensures code quality by enforcing a minimum test coverage before deployment.

You are required to configure a CI/CD pipeline for your project that will:

1. **Automatically run tests on every:**
   - **push to the repository**
   - **pull request**
2. **Measure test coverage and ensure it meets a minimum threshold:**

- Required coverage: ≥ 80%

3. Block deployment if:
   - any test fails
   - coverage is below 80%
4. Simulate a deployment stage (fake deploy):
   - this stage should only run if all previous checks pass
   - just print something to console if unit-test stage will end successfully

Integration test with real DB (Docker)

Instead of mocks:

- spin up PostgreSQL using Docker
- run real repository

Extend `GetRate`:

- retry on failure (e.g., 3 times)

Test:

- first request fails
- second succeeds

Implement Context Support

Modify service function like this:

```
func (s *UserService) GetUserByID(ctx context.Context, id int)
```

Test:

- cancelled context
- timeout